In [ ]:
```
import marimo as mo
from random import Random
import pandas as pd
import matplotlib.pyplot as plt
from pysmt.shortcuts import (
    Symbol, And, Or, Not, Iff, Equals, Ite, Plus, Int, Solver
)
from pysmt.typing import BOOL

MODO = "sync"
```

1. Modelo FSM

Game of Life com espaço de estados finito

O autómato é modelado como um *first-order transition system* (FOTS): um predicado `init` sobre o estado inicial e um predicado `trans` sobre o par (pré-estado, pós-estado). Primeiro formula-se a relação de transição em lógica; depois implementa-se em pySMT, com o modo de transição (síncrono ou assíncrono) escolhido pela constante `MODO`, definida no início do notebook. A estrutura segue a da ficha 7 (FOTS, `declare` / `init` / `trans`, `gera_traco` e `bmc_always`) e, na *k*-indução, a da ficha 8 (`kinduction_always`); os exercícios de origem são indicados ao longo do texto.

Formulação lógica

Estado. Cada célula é uma variável booleana $c_{i,j}$ (e $c'_{i,j}$ no pós-estado), com $0 \leq i, j \leq N+1$. Os três tipos de célula são:

- normais: $\mathcal{N} = \{1..N\}^2$
- iniciais: $\mathcal{I} = \{(i,j) \in \{0..N\}^2 : i = 0 \vee j = 0\}$, em número $2N+1$
- finais: $\mathcal{F} = \{(i,j) \in \{0..N+1\}^2 : i = N+1 \vee j = N+1\}$, em número $2N+3$ (não há células iniciais e finais em comum)

Regra B3|S23. Seja $V(i,j)$ o conjunto das 8 vizinhas de (i,j) e $n(i,j) = \sum_{(u,v)\in V(i,j)} [c_{u,v}]$ o número de vizinhas vivas:

$$R(i,j) \;\equiv\; n(i,j) = 3 \;\vee\; \big(c_{i,j} \wedge n(i,j) = 2\big)$$

Manutenção da fronteira (igual nos dois modos):

$$Fr \;\equiv\; \bigwedge_{(i,j)\in\mathcal{I}} \big(c'_{i,j} \leftrightarrow c_{i,j}\big) \;\wedge\; \bigwedge_{(i,j)\in\mathcal{F}} \neg c'_{i,j}$$

Modo síncrono (todas as células normais transitam em simultâneo):

$$T_{sync} \equiv Fr \wedge \bigwedge_{(i,j)\in\mathcal{N}} \left(c'_{i,j} \leftrightarrow R(i,j)\right)$$

Modo assíncrono (uma e só uma célula normal transita). Cada disjunto é um caso, com a sua atualização e manutenção:

$$T_{async} \equiv Fr \wedge \bigvee_{(a,b)\in\mathcal{N}} \Big(\underbrace{\left(c'_{a,b} \leftrightarrow R(a,b)\right)}_{\text{Atualização}} \wedge \underbrace{\bigwedge_{(i,j)\in\mathcal{N}\setminus\{(a,b)\}} \left(c'_{i,j} \leftrightarrow c_{i,j}\right)}_{\text{Manutenção}} \Big)$$

A estrutura de cada caso (pré-condição, atualização, manutenção) é a do exercício 2 da ficha 7. A pré-condição é vazia: a célula escolhida pode ficar com o mesmo valor (*stuttering*). É o comportamento que o enunciado descreve, porque a regra B3|S23 inclui como resultados possíveis "mantém-se viva" e "continua morta"; escolher uma célula cuja regra dá o valor atual é, por isso, uma transição válida. Isto mantém ainda a relação total (sem deadlocks). A variante estrita acrescentaria a cada disjunto a pré-condição $R(a,b) \nleftrightarrow c_{a,b}$.

In [ ]:
```python
def declare(t, N):
    if not N > 16:
        raise ValueError("N must be greater than 16")
    return {
        (i, j): Symbol(f"cell_{i}_{j}_{t}", BOOL)
        for i in range(0, N + 2)
        for j in range(0, N + 2)
    }
```

In [ ]:
```python
def iniciais(N):
    return [(i, 0) for i in range(N + 1)] + [(0, j) for j in range(1, N + 1)]
```

In [ ]:
```python
def finais(N):
    return [(i, N + 1) for i in range(N + 2)] + [(N + 1, j) for j in range(N + 1)]
```

In [ ]:
```python
def normais(N):
    return [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
```

In [ ]:
```python
def init(state, N):
    constraints = []
    for (i, j) in normais(N):
        if i <= 3 and j <= 3:
            constraints.append(state[(i, j)])
        else:
            constraints.append(Not(state[(i, j)]))
    return And(constraints)
```

In [ ]:
```python
def initial_final_cells(state, N, seed=0):
    rng = Random(seed)
    constraints = []

    for c in iniciais(N):
        constraints.append(state[c] if rng.random() < 0.5 else Not(state[c]))
```

```
    for c in finais(N):
        constraints.append(Not(state[c]))

    return And(constraints)
```

In [ ]:
```
def vizinhos(i, j):
    return [
        (i + di, j + dj)
        for di in (-1, 0, 1)
        for dj in (-1, 0, 1)
        if (di, dj) != (0, 0)
    ]
```

In [ ]:
```
def regra(curr, i, j):
    n = Plus([Ite(curr[v], Int(1), Int(0)) for v in vizinhos(i, j)])
    return Or(Equals(n, Int(3)), And(curr[(i, j)], Equals(n, Int(2))))
```

In [ ]:
```
def fronteira(curr, prox, N):
    return And(
        [Iff(prox[c], curr[c]) for c in iniciais(N)]
        + [Not(prox[c]) for c in finais(N)]
    )
```

In [ ]:
```
def trans_sync(curr, prox, N):
    return And(
        fronteira(curr, prox, N),
        And([Iff(prox[(i, j)], regra(curr, i, j)) for (i, j) in normais(N)]),
    )
```

In [ ]:
```
def trans_async(curr, prox, N):
    cel = normais(N)
    casos = []
    for a in cel:
        atualizacao = Iff(prox[a], regra(curr, *a))
        manutencao = And([Iff(prox[c], curr[c]) for c in cel if c != a])
        casos.append(And(atualizacao, manutencao))
    return And(fronteira(curr, prox, N), Or(casos))
```

In [ ]:
```
def trans(curr, prox, N, modo=MODO):
    if modo == "sync":
        return trans_sync(curr, prox, N)
    if modo == "async":
        return trans_async(curr, prox, N)
    raise ValueError("modo deve ser 'sync' ou 'async'")
```

2. Geração de traços

In [ ]:
```
def fots(N, modo=MODO, seed=0):
    def declare_(i):
        return declare(i, N)
```

```
    def init_(state):
        return And(init(state, N), initial_final_cells(state, N, seed))

    def trans_(curr, prox):
        return trans(curr, prox, N, modo)

    return declare_, init_, trans_
```

In [ ]:
```
def grelha_modelo(model, state):
    N = max(i for (i, _) in state) - 1
    return [
        [1 if model.get_py_value(state[(i, j)]) else 0 for j in range(N + 2)]
        for i in range(N + 2)
    ]
```

In [ ]:
```
def grelha_str(grelha):
    N = len(grelha) - 2

    def linha(i):
        r = grelha[i]
        return (
            str(r[0]) + " | "
            + " ".join(str(v) for v in r[1:N + 1])
            + " | " + str(r[N + 1])
        )
    larg = len(linha(1))
    return "\n".join(
        [linha(0), "-" * larg]
        + [linha(i) for i in range(1, N + 1)]
        + ["-" * larg, linha(N + 1)]
    )
```

In [ ]:
```
def mostra(grelha):
    print(grelha_str(grelha))
```

In [ ]:
```
def gera_traco(declare, init, trans, k, verbose=True):
    with Solver(name="z3") as s:
        states = [declare(i) for i in range(k)]

        constraints = []

        constraints.append(init(states[0]))

        for i in range(k - 1):
            constraints.append(trans(states[i], states[i + 1]))

        s.add_assertion(And(constraints))

        if s.solve():
            model = s.get_model()
            traco = [grelha_modelo(model, st) for st in states]
            if verbose:
                for i, grelha in enumerate(traco):
                    print(f"Estado {i}:")
                    mostra(grelha)
```

```
                    print()
            return traco
        if verbose:
            print("Não foi encontrado um traço válido")
        return None
```

3. Simulação e validação

In [ ]:
```
def regra_py(g, i, j):
    n = 0
    for di in (-1, 0, 1):
        for dj in (-1, 0, 1):
            if (di, dj) != (0, 0):
                n += g[i + di][j + dj]
    return 1 if n == 3 or (g[i][j] == 1 and n == 2) else 0
```

In [ ]:
```
def passo_sync_py(g, N):
    nova = [linha[:] for linha in g]
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            nova[i][j] = regra_py(g, i, j)
    return nova
```

In [ ]:
```
def estado_inicial_py(N, seed=0):
    rng = Random(seed)
    g = [[0] * (N + 2) for _ in range(N + 2)]
    for (i, j) in iniciais(N):
        g[i][j] = 1 if rng.random() < 0.5 else 0
    for i in range(1, 4):
        for j in range(1, 4):
            g[i][j] = 1
    return g
```

In [ ]:
```
def trajetoria_sync(N, seed=0, limite=1_000_000):
    g = estado_inicial_py(N, seed)
    visto = {tuple(map(tuple, g)): 0}
    estados = [g]
    while len(estados) < limite:
        g = passo_sync_py(g, N)
        chave = tuple(map(tuple, g))
        if chave in visto:
            return visto[chave], estados
        visto[chave] = len(estados)
        estados.append(g)
    return None, None
```

In [ ]:
```
def valida_traco(traco, N, modo):
    ini, fin = set(iniciais(N)), set(finais(N))
    for t in range(len(traco) - 1):
        atual, seguinte = traco[t], traco[t + 1]
        for (i, j) in ini:
            if seguinte[i][j] != atual[i][j]:
```

```
                return False, f"passo {t}→{t+1}: célula inicial ({i},{j}) mudou"
        for (i, j) in fin:
            if seguinte[i][j] != 0:
                return False, f"passo {t}→{t+1}: célula final ({i},{j}) está viva"
        if modo == "sync":
            esperado = passo_sync_py(atual, N)
            for (i, j) in normais(N):
                if seguinte[i][j] != esperado[i][j]:
                    return False, f"passo {t}→{t+1}: célula ({i},{j}) devia ser
{esperado[i][j]}"
        else:
            mudou = [(i, j) for (i, j) in normais(N) if seguinte[i][j] != atual[i]
[j]]
            if len(mudou) > 1:
                return False, f"passo {t}→{t+1}: mudaram {len(mudou)} células
normais"
            if mudou:
                i, j = mudou[0]
                if seguinte[i][j] != regra_py(atual, i, j):
                    return False, f"passo {t}→{t+1}: ({i},{j}) não segue a regra"
    return True, f"{len(traco) - 1} passos verificados"
```

4. Verificação formal

In [ ]:
```
def erro(state, N):
    return And([Not(state[c]) for c in normais(N)])
```

In [ ]:
```
def sem_erro(N):
    return lambda state: Not(erro(state, N))
```

In [ ]:
```
def invariante_inducao(N):
    return lambda state: And(
        Not(erro(state, N)),
        And([Not(state[c]) for c in finais(N)]),
    )
```

In [ ]:
```
def bmc_always(declare, init, trans, inv, K):
    for k in range(1, K + 1):
        with Solver(name="z3") as s:
            states = [declare(i) for i in range(k)]
            constraints = []

            constraints.append(init(states[0]))

            for i in range(k - 1):
                constraints.append(trans(states[i], states[i + 1]))

            constraints.append(Not(inv(states[-1])))

            s.add_assertion(And(constraints))

            if s.solve():
                model = s.get_model()
```

```
                return [grelha_modelo(model, st) for st in states]
    return None
```

In [ ]:

```
def kinduction_always(declare, init, trans, inv, k):

    with Solver(name="z3") as solver:

        states = [declare(i) for i in range(k)]

        base = [init(states[0])]


        for i in range(k - 1):

            base.append(trans(states[i], states[i + 1]))


        caso_base = And(And(base), Or([Not(inv(s)) for s in states]))

        solver.add_assertion(caso_base)


        if solver.solve():

            model = solver.get_model()

            return "caso base", [grelha_modelo(model, s) for s in states]


        solver.reset_assertions()


        states = [declare(i) for i in range(k + 1)]

        inductive = []

        for i in range(k):

            inductive.append(trans(states[i], states[i + 1]))


        inductive.append(And([inv(states[i]) for i in range(k)]))

        inductive.append(Not(inv(states[k])))


        step_case = And(inductive)

        solver.add_assertion(step_case)
```

```
        if solver.solve():

            model = solver.get_model()

            return "passo indutivo", [grelha_modelo(model, s) for s in states]



    return None, None
```

In [ ]:
```
def alguma_constante(N):
    cel = normais(N)
    return lambda estados: Or([
        Or(And([st[c] for st in estados]), And([Not(st[c]) for st in estados]))
        for c in cel
    ])
```

In [ ]:
```
def bmc_eventually(declare, init, trans, constante, bound):
    for k in range(1, bound + 1):
         with Solver(name="z3") as s:
            states = [declare(i) for i in range(k)]
            constraints = []

            constraints.append(init(states[0]))

            for i in range(k - 1):
                constraints.append(trans(states[i], states[i + 1]))

            ciclos = [
                And(trans(states[-1], states[j]), constante(states[j:]))
                for j in range(k)
            ]
            constraints.append(Or(ciclos))

            s.add_assertion(And(constraints))

            if s.solve():
                model = s.get_model()
                j = next(j for j in range(k) if model.get_py_value(ciclos[j]))
                return [grelha_modelo(model, st) for st in states], j

    return None, None
```

5. Análise de propriedades

In [ ]:
```
def orbita_sync(N, seed=0):
    inicio, estados = trajetoria_sync(N, seed)
    return {
        "pre-período": inicio,
        "período": len(estados) - inicio,
```

```
        "estados distintos": len(estados),
        "erro na órbita": any(
            all(g[i][j] == 0 for (i, j) in normais(N)) for g in estados
        ),
    }
```

In [ ]:
```
def ciclo_sync(N, seed=0):
    inicio, estados = trajetoria_sync(N, seed)
    return inicio, estados[inicio:]
```

In [ ]:
```
def classifica_celulas(ciclo, N):
    cel = normais(N)
    mortas = [(i, j) for (i, j) in cel if all(g[i][j] == 0 for g in ciclo)]
    vivas = [(i, j) for (i, j) in cel if all(g[i][j] == 1 for g in ciclo)]
    return mortas, vivas
```

In [ ]:
```
def mudancas(traco, N):
    return [
        (t + 1, (i, j), traco[t][i][j], traco[t + 1][i][j])
        for t in range(len(traco) - 1)
        for (i, j) in normais(N)
        if traco[t][i][j] != traco[t + 1][i][j]
    ]
```

In [ ]:
```
def repete_sequencia(N, seed, sequencia):
    g = estado_inicial_py(N, seed)
    for t, ((i, j), depois) in enumerate(sequencia, start=1):
        if regra_py(g, i, j) != depois or g[i][j] == depois:
            return False, f"passo {t}: a célula ({i},{j}) não passa a {depois}
pela regra"
        g[i][j] = depois
    instaveis = [(i, j) for (i, j) in normais(N) if regra_py(g, i, j) != g[i][j]]
    if instaveis:
        return False, f"o estado final não é estável ({len(instaveis)} células
mudariam)"
    return True, [(i, j) for (i, j) in normais(N) if g[i][j]]
```

In [ ]:
```
def tabela_validacao(N):
    linhas = []
    for tipo, k in (("sync", 20), ("async", 6)):
        traco = gera_traco(*fots(N, tipo), k + 1, verbose=False)
        ok, msg = valida_traco(traco, N, tipo)
        linhas.append(f"| {tipo} | {k} | {'✓ ok' if ok else '✗ falhou'} | {msg}
|")
    return mo.md("| modo | k | validação | detalhe |\n|---|---|---|---|\n" +
"\n".join(linhas))
```

In [ ]:
```
def prop1_sync(N):

    orb = orbita_sync(N, seed=0)

    K = orb["estados distintos"]

    traco = bmc_always(*fots(N, "sync"), sem_erro(N), K)
```

```
    tab = pd.DataFrame(
        [{"semente": sd, **orbita_sync(N, seed=sd)} for sd in range(10)]
    )
    if traco is None:
        veredito = f"O BMC não encontrou contra-exemplo até d={K - 1}. Sendo K o tamanho total da órbita, prova-se por exaustão que o estado de erro é inalcançável (para a semente 0)."
    else:
        veredito = f"Contra-exemplo no modo síncrono atingido em d={len(traco) - 1} passos."

    return mo.vstack([
        mo.md(f"### Modo síncrono (Prova por Exaustão)\n\n{veredito}\n\nA tabela demonstra a estabilização para outras sementes:"),
        mo.ui.table(tab, selection=None),
    ])
```

In [ ]:

```
def prop1_inducao(N):
    linhas = []
    cti1 = None
    for k in (1, 2, 3):
        onde, ce = kinduction_always(*fots(N, "sync"), invariante_inducao(N), k)
        if k == 1 and onde == "passo indutivo":
            cti1 = ce
        linhas.append({
            "k": k,
            "resultado": f"falha no {onde}" if onde else "válida por k-indução",
        })
    texto = ""
    if cti1 is not None:
        texto = "\n\nO invariante falha no passo indutivo. O solver encontra
```

```
estados (mesmo que inalcançáveis a partir da origem) onde células vivas transitam
todas para zero num só passo, mostrando que a propriedade não é k-indutiva para
estes valores."



    return mo.vstack([

        mo.md("### Análise por k-indução"),

        mo.ui.table(pd.DataFrame(linhas), selection=None),

        mo.md(texto),

    ])
```

In [ ]:

```
def prop1_async(N):

    traco = bmc_always(*fots(N, "async"), sem_erro(N), 13)

    if traco is None:

        return mo.md("Sem contra-exemplo até d=12 (assíncrono).")



    d = len(traco) - 1

    tab = pd.DataFrame([

        {"passo": passo, "célula": f"({cel[0]},{cel[1]})", "antes": antes,
"depois": depois}

        for (passo, cel, antes, depois) in mudancas(traco, N)

    ])



    return mo.vstack([

        mo.md(f"### Modo assíncrono (Contra-exemplo)\n\nA propriedade falha no
modo assíncrono. O BMC gerou um traço de d={d} passos (contra-exemplo mínimo) onde
o estado de erro é forçado, alterando uma célula por passo:"),

        mo.ui.table(tab, selection=None)

    ])
```

In [ ]:

```
def prop2_sync(N):

    linhas = []

    for sd in range(10):
```

```
        pre, ciclo = ciclo_sync(N, seed=sd)

        mortas, vivas = classifica_celulas(ciclo, N)

        linhas.append({

            "semente": sd,

            "pré-período": pre,

            "período": len(ciclo),

            "sempre mortas": len(mortas),

            "sempre vivas": len(vivas)

        })



    return mo.vstack([

        mo.md("### Modo síncrono\n\nA propriedade falha. O sistema tende a
estabilizar, com a esmagadora maioria das células a fixarem-se num estado
permanente (ver tabela). Este comportamento foi confirmado pelo BMC de ciclo para
as primeiras sementes."),

        mo.ui.table(pd.DataFrame(linhas), selection=None)

    ])
```

In [ ]:
```
def prop2_async_sem_fairness(N):

    return mo.md("### Modo assíncrono (sem fairness)\n\nA propriedade é
trivialmente falsa, pois a relação de transição permite *stuttering*. O BMC
encontra um ciclo com 1 estado (onde o sistema fica parado), tornando todas as
células permanentes.")
```

In [ ]:
```
def prop2_async_com_fairness(N):

    sequencia = [

        ((2, 3), 0), ((2, 2), 0), ((1, 8), 1), ((2, 1), 0), ((1, 9), 1),

        ((4, 2), 1), ((3, 1), 0), ((4, 3), 1), ((1, 8), 0),

    ]

    ok, res = repete_sequencia(N, 0, sequencia)



    texto = "O BMC encontrou uma sequência de 9 passos que conduz a um estado
estável, invalidando a propriedade mesmo assumindo execução justa." if ok else
f"Falha na validação: {res}"
```

```
    return mo.vstack([
        mo.md(f"### Modo assíncrono (com fairness)\n\n{texto}"),
        mo.ui.table(
            pd.DataFrame([{"passo": p, "célula": f"({c[0]},{c[1]})", "novo valor":
v}
                for p, (c, v) in enumerate(sequencia, start=1)]),
            selection=None,
        ),
    ])
```

6. Visualização e interface

Geração de traços

Mostra um traço de 5 passos para $N = 17$ (semente 0), gerado com `gera_traco(..., 6)`: tal como na ficha, o último argumento é o número de estados, ou seja, $k - 1$ passos. O modo de transição começa com o valor de `MODO` e pode trocar-se aqui. Em cada estado, a linha de cima e a coluna da esquerda são as células iniciais (constantes), a linha de baixo e a coluna da direita são as finais (sempre mortas), e o bloco do meio são as $N \times N$ células normais.

A animação da geração de traços foi gerada pelo chatGPT.

In [ ]:
```
modo = mo.ui.dropdown(
    options=["sync", "async"], value=MODO, label="Modo de transição"
)
modo
```

Modo de transição sync

In [ ]:
```
# A geração do traço mantém-se independente da animação.
# verbose=False evita imprimir todas as grelhas em texto.
traco_anim = gera_traco(*fots(17, modo.value), 6, verbose=False)
```

In [ ]:
```
# Estado reativo: índice da grelha atualmente apresentada.
get_indice, set_indice = mo.state(0)
```

In [ ]:
```
# O próprio controlo refresh inicia/pausa a reprodução.
# O menu permite escolher o intervalo entre estados.
```

```
reproducao = mo.ui.refresh(
    options=[0.1, 0.2, 0.5, 1.0],
    default_interval=0.2,
    label="Reproduzir / Pausar — intervalo",
)
reiniciar = mo.ui.button(
    label="Reiniciar",
    on_click=lambda _: set_indice(0),
)
mo.hstack([reproducao, reiniciar], justify="start", align="center")
```

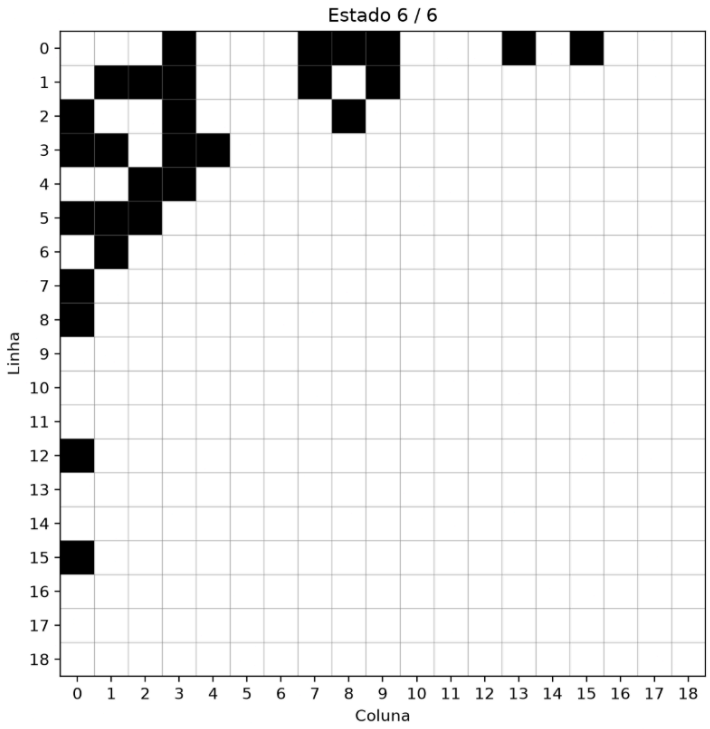
Reproduzir / Pausar — intervalo off Reiniciar

In [ ]:
```
# Cada atualização do temporizador avança exatamente um estado.
# O índice fica limitado ao último estado do traço.
if reproducao.value and traco_anim:
    set_indice(
        lambda atual: min(atual + 1, len(traco_anim) - 1)
    )
```

In [ ]:
```
if not traco_anim:
    saida = mo.md("**Não foi encontrado um traço válido para visualizar.**")
else:
    indice = min(get_indice(), len(traco_anim) - 1)
    grelha = traco_anim[indice]
    altura = len(grelha)
    largura = len(grelha[0])

    # A figura adapta-se às dimensões da grelha recebida.
    fig, ax = plt.subplots(
        figsize=(min(12, max(4, largura * 0.35)),
                 min(12, max(4, altura * 0.35)))
    )
    ax.imshow(grelha, cmap="binary", vmin=0, vmax=1,
              interpolation="nearest")
    ax.set_xticks(range(largura))
    ax.set_yticks(range(altura))
    ax.set_xticks([x - 0.5 for x in range(1, largura)], minor=True)
    ax.set_yticks([y - 0.5 for y in range(1, altura)], minor=True)
    ax.grid(which="minor", color="gray", linewidth=0.5, alpha=0.6)
    ax.tick_params(which="minor", bottom=False, left=False)
    ax.set_xlim(-0.5, largura - 0.5)
    ax.set_ylim(altura - 0.5, -0.5)
    ax.set_title(f"Estado {indice + 1} / {len(traco_anim)}")
    ax.set_xlabel("Coluna")
    ax.set_ylabel("Linha")
    fig.tight_layout()

    saida = mo.vstack([
        mo.md(f"**Estado {indice + 1} de {len(traco_anim)}**"),
        fig,
    ])
    plt.close(fig)
saida
```



7. Execução e apresentação dos resultados

Validação Cruzada

Para garantir a correção do modelo SMT, implementou-se um simulador determinístico em Python puro (`regra_py` , `passo_sync_py`). A função `valida_traco` verifica a integridade dos traços gerados pelo z3, garantindo que:

- As células iniciais e finais mantêm as restrições de fronteira.
- **Modo síncrono:** As transições correspondem exatamente à matriz calculada pelo simulador.
- **Modo assíncrono:** Ocorre, no máximo, a transição de uma única célula normal por passo (ou *stuttering*), respeitando a regra B3|S23.

In [ ]:
```
tabela_validacao(17)
```

modo k validação detalhe sync 20 ✔ ok 20 passos verificados async 6 ✔ ok 6 passos verificados

Propriedade 1: Invariante de Segurança

Propriedade: O estado de erro (todas as células normais mortas) nunca é alcançado. Formalmente: $G \neg erro$.

Estratégia de Verificação:

- **Modo síncrono:** Como o sistema é determinístico e finito, utiliza-se BMC iterativo para provar a propriedade por exaustão (cobrindo o tamanho total da órbita) e k-indução para tentar a verificação generalizada no tempo.
- **Modo assíncrono:** Utiliza-se BMC convencional para procurar um contra-exemplo até um limite de profundidade K.

Foi utilizado o ChatGPT para a opção de execução da prova formal síncrona por exaustão.

In [ ]:
```
correr_teste = mo.ui.checkbox(value=False, label="Executar prova formal síncrona no z3 (Aviso: demora ~18 minutos)")
correr_teste
```

Executar prova formal síncrona no z3 (Aviso: demora ~18 minutos)

In [ ]:
```
if correr_teste.value:
    # Executa a prova pesada apenas se a checkbox for ativada
    resultado_prop1 = prop1_sync(17)
else:
    # Resultados guardados (cache) correspondentes ao seu output
    tab_hardcoded = pd.DataFrame([
        {"semente": 0, "pre-período": 156, "período": 1, "estados distintos": 157, "erro na órbita": False},
        {"semente": 1, "pre-período": 51, "período": 7, "estados distintos": 58, "erro na órbita": False},
        {"semente": 2, "pre-período": 9, "período": 7, "estados distintos": 16, "erro na órbita": False},
        {"semente": 3, "pre-período": 120, "período": 1, "estados distintos": 121, "erro na órbita": False},
        {"semente": 4, "pre-período": 41, "período": 6, "estados distintos": 47, "erro na órbita": False},
        {"semente": 5, "pre-período": 6, "período": 1, "estados distintos": 7, "erro na órbita": False},
        {"semente": 6, "pre-período": 31, "período": 1, "estados distintos": 32, "erro na órbita": False},
        {"semente": 7, "pre-período": 1023, "período": 6, "estados distintos": 1029, "erro na órbita": False},
        {"semente": 8, "pre-período": 998, "período": 1, "estados distintos": 999, "erro na órbita": False},
        {"semente": 9, "pre-período": 774, "período": 1, "estados distintos": 775, "erro na órbita": False}
    ])

    veredito = "O BMC não encontrou contra-exemplo até d=156. Sendo K o tamanho total da órbita, prova-se por exaustão que o estado de erro é inalcançável (para a semente 0)."

    resultado_prop1 = mo.vstack([
        mo.md(f"### Modo síncrono (Prova por Exaustão)\n\n{veredito}\n\nA tabela
```

```
demonstra a estabilização para outras sementes:\n\n*(Resultados em cache. Para executar a prova formal no z3, assinale a caixa acima)*"),
        mo.ui.table(tab_hardcoded, selection=None)
    ])

resultado_prop1
```

O BMC não encontrou contra-exemplo até d=156. Sendo K o tamanho total da órbita, prova-se por exaustão que o estado de erro é inalcançável (para a semente 0).

A tabela demonstra a estabilização para outras sementes:

(Resultados em cache. Para executar a prova formal no z3, assinale a caixa acima)

Search...

semente int64	pre-período int64	período int64	estados distintos int64	erro na órbita bool
0	156	1	157	False
1	51	7	58	False
2	9	7	16	False
3	120	1	121	False
4	41	6	47	False
5	6	1	7	False
6	31	1	32	False
7	1023	6	1029	False
8	998	1	999	False
9	774	1	775	False

10 rows, 5 columns — *No selection*

In [ ]:
```
prop1_inducao(17)
```

Search...

k int64	resultado str
1	falha no passo indutivo
2	falha no passo indutivo
3	falha no passo indutivo

3 rows, 2 columns — *No selection*

O invariante falha no passo indutivo. O solver encontra estados (mesmo que inalcançáveis a partir da origem) onde células vivas transitam todas para zero num só passo, mostrando que a propriedade não é k-indutiva para estes valores.

In [ ]:
```
prop1_async(17)
```

A propriedade falha no modo assíncrono. O BMC gerou um traço de d=9 passos (contra-exemplo mínimo) onde o estado de erro é forçado, alterando uma célula por passo:

Search...

passo int64	célula str	antes int64	depois int64
1	(2,3)	1	0
2	(2,2)	1	0
3	(3,3)	1	0
4	(1,2)	1	0
5	(3,1)	1	0
6	(1,3)	1	0
7	(2,1)	1	0
8	(3,2)	1	0
9	(1,1)	1	0

9 rows, 4 columns

No selection

Conclusão da propriedade 1

modo	resultado	como se obteve
síncrono	o erro **nunca** é alcançado (semente 0)	exaustão: a órbita tem 157 estados e o BMC até $d = 156$ não encontra erro
assíncrono	o erro **é** alcançado, em 9 passos	contra-exemplo do BMC, confirmado por simulação

A diferença vem do tipo de transição. No síncrono há uma única evolução possível, e a partir deste estado inicial ela estabiliza sem passar pelo erro. No assíncrono o solver escolhe a ordem em que as células transitam, e existe uma ordem que mata as 9 células do bloco inicial, uma de cada vez. A propriedade vale então num modo e falha no outro, o que é um resultado válido para o enunciado ("provar ou obter contra-exemplos").

Limitação: a prova do síncrono é para a semente 0 (o estado inicial depende das células iniciais aleatórias). A tabela mostra que as outras sementes testadas também não chegam ao erro, mas para provar o resultado para todas as atribuições iniciais seria preciso deixá-las como variáveis livres.

Propriedade 2: Propriedade de Animação

Propriedade: Nenhuma célula normal permanece infinitamente viva ou morta. Formalmente: $\bigwedge_{c \in \mathcal{N}} \left(G\,F\,c \;\wedge\; G\,F\,\neg c \right)$. A falsificação desta propriedade implica encontrar um traço em ciclo (*lasso*) onde pelo menos uma célula se mantenha constante.

Estratégia de Verificação: Utiliza-se a função `bmc_eventually` para gerar traços até profundidade K terminados num ciclo válido, validando a estabilização das células.

In [ ]:
```
prop2_sync(9)
```

A propriedade falha. O sistema tende a estabilizar, com a esmagadora maioria das células a fixarem-se num estado permanente (ver tabela). Este comportamento foi confirmado pelo BMC de ciclo para as primeiras sementes.

Search...

semente int64	pré-período int64	período int64	sempre mortas int64	sempre vivas int64
0	36	1	78	3
1	328	1	75	6
2	16	126	6	0
3	92	1	73	8
4	6	1	77	4
5	41	1	77	4
6	264	1	77	4
7	598	1	72	9
8	269	1	73	8
9	665	1	74	7

10 rows, 5 columns

No selection

In [ ]:
```
prop2_async_sem_fairness(9)
```

Modo assíncrono (sem fairness)

A propriedade é trivialmente falsa, pois a relação de transição permite *stuttering*. O BMC encontra um ciclo com 1 estado (onde o sistema fica parado), tornando todas as células permanentes.

In [ ]:
```
prop2_async_com_fairness(9)
```

Falha na validação: passo 3: a célula (1,8) não passa a 1 pela regra

Search...

passo int64	célula str	novo valor int64
1	(2,3)	0
2	(2,2)	0
3	(1,8)	1
4	(2,1)	0
5	(1,9)	1
6	(4,2)	1
7	(3,1)	0
8	(4,3)	1
9	(1,8)	0

9 rows, 3 columns

No selection

Conclusão da propriedade 2

modo	resultado	como se obteve
síncrono	**não vale**: quase todas as células ficam constantes no ciclo	análise exata do ciclo (sementes 0 a 9) e *lasso* do BMC (sementes 5 e 2)

modo	resultado	como se obteve
assíncrono, sem *fairness*	**não vale**, de forma trivial	*lasso* com 1 estado (o sistema fica parado)
assíncrono, com *fairness*	**não vale**	estado estável alcançável em 9 passos, onde nenhuma célula muda

A propriedade falha em todos os casos porque o sistema tende a estabilizar: as células longe da atividade ficam mortas para sempre, e as que pertencem a padrões estáveis ficam vivas para sempre. O enunciado aceita este resultado ("provar ou obter contra-exemplos").

Limitações: os resultados do síncrono são para as sementes testadas. No assíncrono, o argumento da *fairness* usa um estado estável, e na variante estrita (a célula escolhida tem de mudar) um estado estável não tem sucessores, por isso esse argumento deixaria de valer e a análise teria de ser refeita.

Análise de Desempenho

1. **Modo Assíncrono:** O tempo de resolução cresce empiricamente a $O(N^{3.6})$, consistente com o modelo teórico de N^4 gerado pela disjunção massiva da relação de transição (manutenção de $N^2 - 1$ células por caso).
2. **Modo Síncrono:** O crescimento escalar varia entre N^2 e N^3, com tempos de resolução residuais (inferiores a 3s para $N = 24, k = 10$).
3. **Escalabilidade em k:** O custo associado à profundidade no modo assíncrono exibe um crescimento super-linear (fator $\times 8.9$ entre $k = 2$ e $k = 10$ para $N = 17$), limitando a aplicabilidade prática do BMC neste modo para K elevados. Uma otimização futura recomendada seria a adoção de restrições *exactly-one* para seletores de transição em vez da enumeração disjuntiva atual.

Search...

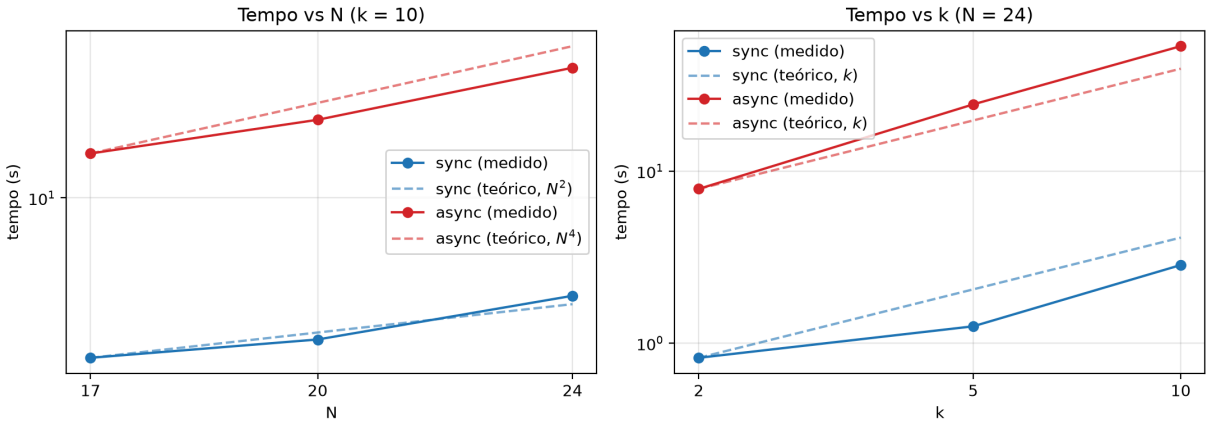
N int64	k int64	sync (s) float64	sync esperado (s) float64	sync medido/esperado float64	async (s) float64	async esperado (s) float64	async medido/esperado float64	async/ float64
17	2	0,22	0,22	1,00	1,99	1,99	1,00	
17	5	0,51	0,54	0,93	7,24	4,97	1,46	
17	10	1,28	1,08	1,18	17,68	9,94	1,78	
20	2	0,46	0,30	1,54	3,79	3,81	1,00	
20	5	0,80	0,75	1,06	11,54	9,52	1,21	
20	10	1,62	1,50	1,08	27,27	19,04	1,43	
24	2	0,82	0,43	1,89	7,87	7,90	1,00	
24	5	1,25	1,08	1,15	24,40	19,74	1,24	
24	10	2,84	2,16	1,31	53,21	39,48	1,35	

9 rows, 9 columns — No selection

Search...

modo str	expoente de N (medido) float64	expoente de N (teórico) int64	expoente de k (medido) float64	expoente de k (teórico) int64
sync	2,92	2	0,87	1
async	3,58	4	1,26	1

2 rows, 5 columns — No selection



O que se observou

1. **Assíncrono em N: compatível com N^4.** Com $k = 2$, o tempo medido coincide com o esperado (3,79 s contra 3,81 s em $N = 20$; 7,87 s contra 7,90 s em $N = 24$). Com k maior o expoente ajustado desce para 3,58, porque o custo de k (ponto seguinte) pesa mais nos N pequenos: com $k = 10$ o tempo só multiplica por 3,0 entre $N = 17$ e $N = 24$, quando a teoria dá 3,97.
2. **Assíncrono em k: mais do que linear.** Para $N = 17$, o tempo sobe $\times 3{,}6$ de $k = 2$ para $k = 5$ e $\times 8{,}9$ até $k = 10$ (a teoria dá $\times 2{,}5$ e $\times 5$); o expoente ajustado é 1,26. A fórmula só cresce linearmente com k, por isso o custo extra deve vir do solver, que tem de escolher em cada passo qual das N^2 células transita (e se há *stuttering*).
3. **Síncrono: dados pouco conclusivos.** Os tempos ficam entre 0,2 s e 2,8 s e variações de décimas de segundo alteram muito as razões (`medido/esperado` entre 0,93 e 1,89).

Os expoentes ajustados (2,92 para N, 0,87 para k) são pouco fiáveis; só se pode dizer que o crescimento é suave, entre N^2 e N^3.

4. **Assíncrono contra síncrono.** O assíncrono é entre 8 e 20 vezes mais lento, e a diferença tende a aumentar com N e k (coerente com o fator N^2 a mais na fórmula).
5. **Consequência.** O BMC repete a verificação para $k = 1, 2, \ldots$ e a propriedade 2 acrescenta restrições de ciclo, por isso o modo assíncrono vai ser o gargalo. A alternativa é uma codificação com variáveis seletoras (uma por célula e por passo, com um *exactly-one*), que torna a fórmula linear em N^2 por passo.

Limitações: uma execução por ponto, uma só semente e uma só máquina; os tempos pequenos são ruidosos e o tempo esperado depende de uma única medição base.